

PSCAN Parallel Structural Clustering for Large Networks

Shikha Mallick*, Nidita Roy[†], Hoksolinvan Chhun[‡], Chris Warren[§]

*shikhamallick@uvic.ca, [†]niditaroy@uvic.ca, [‡]hoksolinvanchhun@uvic.ca, [§]warrenncm@uvic.ca

^{*†‡§}Department of Computer Science

University of Victoria

Victoria, BC, Canada

Abstract—This project studies and reproduces PSCAN, a parallel structural clustering algorithm for large networks in MapReduce. In line with the project requirements, our work addresses all three major phases of the assignment: paper understanding and explanation, implementation, and experimentation on real or large-scale datasets. We first explain the core PSCAN pipeline, including structural similarity computation, graph pruning, hub/outlier detection, and label-propagation-based connected-component discovery. We then implement these components in Python using a modular pipeline for dataset preparation, similarity computation, clustering, and node classification. Finally, we evaluate the implementation using synthetic LFR benchmark graphs for clustering accuracy, Barabási–Albert graphs for runtime and scalability analysis, and a preprocessed Orkut network as a real-data case study. Our reproduced LFR results are close to those reported in the original paper, with strong ARI and NMI values across all six graph sizes, while the runtime experiments show that the implementation scales with additional computational resources. Overall, the project demonstrates that PSCAN remains an effective framework for large-network clustering and provides a practical reproduction of its key ideas in a modern Python-based environment. The code is available here: <https://github.com/chris-warren/CSC-502-PSCAN.git>.

Index Terms—clustering, parallel computing, graphs

I. INTRODUCTION

The analysis of large graphs is a central problem in big data, especially for social and information networks containing millions of vertices and edges. The original PSCAN paper [1] addresses this challenge by proposing a parallel structural clustering algorithm in MapReduce that combines local structural similarity, edge pruning, connected-component discovery, and hub/outlier identification. Rather than directly parallelizing SCAN through repeated breadth-first expansion, PSCAN first computes structural similarities for all edges, removes weak structural connections, and then identifies clusters as connected components in the pruned graph. This reformulation reduces the number of expensive distributed iterations and makes the method suitable for large networks. The original paper evaluates PSCAN on both synthetic benchmark graphs and very large real networks, with accuracy measured using ARI and NMI and efficiency measured through speedup, scaleup, and sizeup experiments.

This project was framed as an exploratory research assignment with three main expectations: (1) *paper understanding and*

explanation, (2) *implementation of the main algorithm or a justified subset*, and (3) *experimentation using real datasets*. Our report is organized to explicitly satisfy these three requirements. First, we explain the PSCAN method and its major subroutines, including parallel calculation of structural similarity, pruning, hub/outlier detection, and label propagation for connected components. Second, we implement the main PSCAN pipeline in Python, together with dataset-generation and preprocessing utilities. Third, we experimentally evaluate the implementation on synthetic and real graph inputs, following the evaluation logic of the original paper as closely as possible within our computational environment.

To support experimentation, we prepared both synthetic and real datasets in a common adjacency-list format. Following the original PSCAN study, we use LFR benchmark graphs to evaluate clustering accuracy because they contain planted community structure, and we use large Barabási–Albert graphs to study runtime behavior and scalability. In addition, we preprocess the Orkut social-network graph as a real-data input and convert its overlapping community memberships into a deterministic single-label approximation for compatibility with our evaluation pipeline. The resulting dataset preparation stage writes adjacency lists, label files when available, metadata summaries, and a global manifest for reproducibility.

Our experiments pursue the same two main goals emphasized in the reference paper: clustering accuracy and runtime performance. For the LFR datasets, we test multiple values of the structural-similarity threshold ϵ , select the value that yields the highest modularity, and then compare our ARI and NMI values with those reported in the paper. For runtime analysis, we evaluate PSCAN on Barabási graphs using multiple CPU configurations to measure the effect of additional compute resources. These experiments allow us to assess both how faithfully the implementation reproduces the reported behavior of PSCAN and where practical differences emerge between our system and the system used in the original paper. For the Orkut network, we observed a network modularity score of

II. DATASET PREPARATION

A. Overview

The dataset preparation stage produces all graph inputs used in our PSCAN experiments and converts them into a common adjacency-list format. The implementation supports both synthetic and real datasets. For synthetic evaluation, we generate LFR benchmark graphs for clustering-accuracy experiments and Barabási–Albert (BA) graphs for runtime analysis using the Python library NetworkX [2]. For real-data experiments, we preprocess the Orkut social-network dataset [3] into the same adjacency-list representation. The pipeline also writes label files when applicable, computes summary statistics, and stores per-dataset metadata together with a global manifest for reproducibility.

B. Synthetic Datasets

1) *LFR Benchmark Graphs*: We use LFR benchmark graphs to evaluate clustering accuracy because they contain planted community structure. The generator accepts the graph size together with standard LFR parameters listed in the Algorithm 1. The values of these parameters were tuned to match the number of nodes, edges and communities in the original PSCAN paper as best as possible. The final values used for these parameters are given in the Table I. After generation, the graph is converted to a simple undirected graph by removing self-loops and collapsing any non-simple representation. Ground-truth node labels are extracted directly from the LFR `community` attribute. Because the returned attribute may be represented in different container forms, the implementation canonicalizes each node’s planted community and maps distinct communities to integer labels. This yields a single-label ground-truth file for each LFR graph, which can be used in downstream evaluation. The prepared LFR datasets are written as adjacency-list files, label files, and metadata JSON records.

2) *Barabási–Albert Graphs*: We use BA graphs for scalability and runtime experiments. These graphs do not provide trustworthy ground-truth communities, so they are used only as structural inputs for performance analysis. The implementation generates BA graphs at the original paper-scale sizes and initializes them from a complete graph before applying preferential attachment. As in the LFR case, the final graph is represented in adjacency-list form and summarised by metadata such as the number of nodes, the number of edges, density, the number of connected components, and the number of isolated vertices. No label file is produced for BA datasets. The BA graph generator accepts the graph size together with standard BA parameters listed in the Algorithm 1. The values of these parameters were tuned to match the number of nodes and edges in the original PSCAN paper as best as possible. The final values used for these parameters are given in the Table I.

Algorithm 1 LFR Dataset Preparation. G : output graph, n : number of nodes in G , τ_1 : exponent for degree distribution, τ_2 : exponent for community-size distribution, μ : mixing parameter, $\bar{d}$: average node degree, $d_{\max}$: maximum node degree, $c_{\min}$: minimum community size, $c_{\max}$: maximum community size, s : seed.

Input: $n, \tau_1, \tau_2, \mu, \bar{d}, d_{\max}, c_{\min}, c_{\max}, s$

Output: G

for $u \leftarrow 0$ **to** $n - 1$ **do**

Sample node degree: $P(d_u) \propto d^{-\tau_1}; d_u \approx \bar{d} \leq d_{\max}$ with seed s .

while $\sum_i c_i < n$ **do**

Sample community size: $P(c_i) \propto c^{-\tau_2}; c_{\min} \leq c_i \leq c_{\max}$ with seed s .

Assign nodes to communities.

foreach node u **do**

Add $(1 - \mu) \cdot d_u$ intra-community edges.

Add $\mu \cdot d_u$ inter-community edges.

Simplify G to an undirected loop-free graph.

return

Algorithm 2 BA Dataset Preparation. G : output graph, n : number of nodes in G , c_g : number of nodes in the initial complete graph K_{c_g} , s : seed, V : nodeset of G .

Input: n, m, c_g, s

Output: G

Initialize $G \leftarrow K_{c_g}$ with s .

while $|V(G)| < n$ **do**

Add one new node u .

Select m existing nodes with probability proportional to degree with s .

Connect u to the selected nodes.

Simplify G to an undirected loop-free graph.

return

TABLE II
BA DATASET STATISTICS

Dataset	Nodes (n)	Edges	m	seed s
Barabasi-1M	1,000,000	13,999,895	7	10042
Barabasi-2M	2,000,000	27,999,895	7	10043
Barabasi-3M	3,000,000	33,999,847	7	10044
Barabasi-4M	4,000,000	59,999,880	7	10045

C. Real Dataset

1) *Orkut Graph Conversion*: To incorporate a real-world graph, we preprocess the Orkut social-network dataset. The original graph is provided as a text edge list and contains 3,072,441 nodes and 117,185,083 edges. Our conversion script reads the file line by line, skips comment lines, ignores malformed records, removes self-loops, and inserts each valid edge in both directions to build an undirected adjacency structure. After processing all edges, the script writes a sorted

TABLE I
LFR DATASET STATISTICS

Dataset	Nodes (n)	Edges	Clusters	τ_1	τ_2	μ	d	Actual d	d_{max}	c_{min}	c_{max}	seed s
Graph-5k	5,000	35,715	279	3.76	2.366	0.1	14.2504	14.286	40	10	50	5041
Graph-10k	10,000	70,468	598	3.8	2.7	0.1	14.073	14.0936	40	10	50	10041
Graph-20k	20,000	143,698	1007	3.775	2.0	0.1	14.3521	14.3698	40	10	50	20041
Graph-40k	40,000	285,653	2033	3.85	1.99	0.1	14.28265	14.26375	40	10	50	40041
Graph-80k	80,000	576,425	3868	3.84	1.755	0.1	14.410625	14.41605	40	10	50	80041
Graph-160k	160,000	2,301,779	4118	6.8	2.07	0.1	28.7722375	28.6656875	80	20	100	160041

adjacency-list file in the same format used by the synthetic datasets. This ensures that the Orkut graph can be used directly by the PSCAN pipeline without any changes to downstream code.

2) *Single-Label Ground Truth for Orkut*: The Orkut ground-truth community data is overlapping, meaning that a node may belong to multiple communities. Since our evaluation setup expects one label per node, we reduce the overlapping memberships to a single-label approximation. The script first reads all communities, records the set of communities containing each node, and computes community sizes. It then assigns each node to the smallest community containing it, breaking ties by community identifier. Nodes that do not appear in any community are assigned the label “-1”. This heuristic preserves the most specific available membership while still producing a deterministic one-label-per-node ground-truth file.

D. Output Artifacts

For every prepared dataset, the pipeline writes graph data in adjacency-list form and, when available, a tab-separated node-label file. It also computes dataset statistics such as the number of nodes, number of edges, average realized degree, density, number of connected components, largest component size, and isolated vertices. These values, together with generation parameters and file paths, are stored in per-dataset metadata JSON files. Finally, all datasets are listed in a single `manifest.json` file, which serves as a reproducible record of the prepared inputs used in our experiments.

III. METHODOLOGY

A. Structural Similarity Computation

To identify structurally coherent regions in large graphs, we compute the structural similarity between adjacent vertices. Given an undirected graph $G = (V, E)$, the similarity between two connected nodes u and v is defined as:

$$\text{sim}(u, v) = \frac{|N[u] \cap N[v]|}{\sqrt{|N[u]| \cdot |N[v]|}} \quad (1)$$

where $N[u] = N(u) \cup \{u\}$ denotes the closed neighborhood of node u . This formulation captures the normalized overlap between local neighborhoods. The resulting similarity scores are used to prune weak edges using a threshold ϵ , retaining only strong structural connections. The Algorithm 3 gives the step-by-step process.

Algorithm 3 Mapper for Structural Similarity. $Adj(u)$: adjacency list of node u , $N[u]$: closed neighborhood of u , V : node set.

Input: Adj

Output: emitted key-value pairs for structural similarity computation

for each node $u \in V$ **do**

$N[u] \leftarrow Adj(u) \cup \{u\}$

for each neighbor $v \in Adj(u)$ **such that** $u < v$ **do**

Emit (u, v) with $(u, N[u])$

$N[v] \leftarrow Adj(v) \cup \{v\}$

Emit (u, v) with $(v, N[v])$

return

Algorithm 4 Reducer for Structural Similarity. $N[u]$: closed neighborhood of node u , I : intersection size, S : structural similarity score.

Input: grouped records for each edge (u, v)

Output: similarity score for each edge (u, v)

for each edge (u, v) **do**

Retrieve $N[u]$ and $N[v]$

$I \leftarrow |N[u] \cap N[v]|$

$S \leftarrow \frac{I}{\sqrt{|N[u]| \cdot |N[v]|}}$

Output (u, v, S)

return

B. Hub and Outlier Detection

After clustering the pruned graph using label propagation, each node is assigned a structural role based on its connectivity pattern. Let μ denote the minimum number of neighbors required for a node to be considered a core node. Nodes classified as hubs connect multiple clusters, while outliers correspond to sparsely connected nodes. This classification provides additional structural insight beyond cluster assignments. The similarity computation is further parallelized by partitioning the node set across multiple workers, where each worker processes a subset of nodes independently and computes partial similarity records that are later aggregated. The Algorithm 4 gives the step-by-step process.

C. Label Propagation Algorithm

The main algorithm takes as input the pruned graph structure from the previous structural similarity algorithm. It comprises an outer-most while loop maintaining a global status for all

Algorithm 5 Hub and Outlier Detection. $L(u)$: cluster label of node u , $Adj(u)$: pruned adjacency list of u , μ : degree threshold, C : set of neighboring cluster labels, $type(u)$: node type.

Input: L, Adj, μ

Output: $type(u)$ for all $u \in V$

for each node $u \in V$ **do**

if $|Adj(u)| \geq \mu$ **then**
 $type(u) \leftarrow \text{core}$

else

$C \leftarrow \emptyset$

for each neighbor $v \in Adj(u)$ **do**

if $v \in L$ **then**

$C \leftarrow C \cup \{L(v)\}$

if $|C| \geq 2$ **then**

$type(u) \leftarrow \text{hub}$

else

$type(u) \leftarrow \text{outlier}$

return

pruned nodes, indicating whether any node's label (centroid) has been previously modified/updated (True) or not (False). Based on this information, it keeps iterating through every vertex in the algorithm until all labels converge to False, thus indicating that no new updates will continue to happen, and the resulting labels are finalized. The Algorithm 6 gives the step-by-step process.

Algorithm 6 Label Propagation Cluster Convergence (LPCC). $G = (V, E)$: input graph, $N(v)$: neighbors of node v , $\ell_v^{(t)}$: label of node v at iteration t , $changed$: update flag.

Input: $G = (V, E)$ with initial labels ℓ_v for all $v \in V$

Output: final map $v \mapsto \ell_v$

$t \leftarrow 0$

$changed \leftarrow \text{false}$

while $changed = \text{false}$ **do**

for each node $v \in V$ **do**

$\ell_v^{(t+1)} \leftarrow \min(\{\ell_v^{(t)}\} \cup \{\ell_u^{(t)} : u \in N(v)\})$

if $\ell_v^{(t+1)} < \ell_v^{(t)}$ **then**

$changed \leftarrow \text{true}$

$t \leftarrow t + 1$

return

The algorithm attempts to converge to its connected vertices with the minimum vertex ID. Where it repeatedly updates the label for each node until it converges using this operation:

$$\min(\ell_v^{(t)} \cup \{\ell_u^{(t)} : u \in N(v)\}) \quad (2)$$

Our implementation varies from the reference paper's implementation, which involves looping in a similar manner. However, instead, it repeatedly calls the Map-Reduce phase until the list of all label statuses is false. Our implementation

attempts to simplify the Map-Reduce phase as the Reduce phase in the initial paper was ambiguously described and added an extra layer of unnecessary complexity to the overall as outlined by its Map-Reduce phase in the Algorithms 7 and 8.

Algorithm 7 LPCC Mapper. u : input vertex ID, $status(u)$: activation status of u , $\ell(u)$: label of u , $Adj(u)$: adjacency list of u .

Input: vertex ID u and structure information $(status(u), \ell(u), Adj(u))$

Output: key-value pairs $\langle key', value' \rangle$

if $status(u)$ is activated **then**

for each neighbor $v_i \in Adj(u)$ **do**

$key' \leftarrow v_i$.

$value' \leftarrow \ell(u)$.

Emit $\langle key', value' \rangle$

$key' \leftarrow u$.

$value' \leftarrow (status(u), \ell(u), Adj(u))$.

Emit $\langle key', value' \rangle$ **return**

The pseudocode in the Algorithm 7 outlines the mapper of the original reference paper. It takes as input a 2-tuple structured in the format of (key, (status, label, [adjacency list])) and outputs key-value pairs that are either (key, label) or (key, (status, label, [adjacency list])) per vertex for additional processing.

Algorithm 8 LPCC Reducer. u : input vertex ID, $status(u)$: activation status of u , $\ell(u)$: current label of u , $Adj(u)$: adjacency list of u , $\ell_{\min}$: minimum label received from neighbors.

Input: vertex ID u and values containing structure information and neighboring labels

Output: updated key-value pair $\langle key', value' \rangle$

$\ell_{\min} \leftarrow \ell(u)$.

for each value in values do

if value contains structure information **then**

 Retain $(status(u), \ell(u), Adj(u))$.

else

$\ell_{\min} \leftarrow \min(\ell_{\min}, \text{received label})$.

if $\ell_{\min} < \ell(u)$ **then**

$status(u) \leftarrow \text{active}$.

$\ell(u) \leftarrow \ell_{\min}$.

else

$status(u) \leftarrow \text{inactive}$.

$key' \leftarrow u$.

$value' \leftarrow (status(u), \ell(u), Adj(u))$.

Emit $\langle key', value' \rangle$ **return**

The reducer phase in the Algorithm 8 has to manage the extra overhead of handling different key-value structures whilst performing the label propagation logic, thus introducing an extra layer of complexity to manage.

TABLE III

MODULARITY RESULTS FOR LFR DATASETS UNDER DIFFERENT ϵ VALUES USING THE PRUNED PSCAN INTERMEDIATE GRAPHS.

Dataset	$\epsilon = 0.2$	$\epsilon = 0.4$	$\epsilon = 0.6$	$\epsilon = 0.8$	$\epsilon = 1.0$
lfr_5000	0.993397	0.994831	0.995681	0.992484	0.000000
lfr_10000	0.996185	0.997380	0.997963	0.996547	0.666667
lfr_20000	0.998270	0.998670	0.998774	0.997146	0.000000
lfr_40000	0.999067	0.999332	0.999400	0.998670	0.000000
lfr_80000	0.999554	0.999659	0.999671	0.999187	0.857143
lfr_160000	0.999672	0.999718	0.999664	0.999184	0.000000

D. Final Output

Finally, our implementation returns a collection of key-value pairs of each node and their corresponding finalized labels.

IV. RESULTS

A. Experiments

We designed the experiments to replicate the evaluation methodology outlined in the original PSCAN paper to the best of our knowledge. The methodology is split into two sections: clustering accuracy utilizing synthetic graph datasets and runtime performance utilizing BA graphs. The PSCAN pipeline was implemented in Python using a modular design for similarity computation, clustering, and node classification, with core libraries including NumPy, SciPy, NetworkX, and Pandas. Experiments were conducted on the Trillium cluster of Alliance Canada using Python 3.11 within a virtual environment. Each job was allocated a maximum runtime of 4 hours, and experiments were executed with 4, 8, and 15 CPU cores to evaluate scalability. Intermediate data were stored in scratch storage to reduce I/O overhead. The implementation efficiently utilizes available computational resources and is designed to scale to large graph datasets.

1) *Clustering Accuracy*: To evaluate clustering accuracy, we ran PSCAN on 6 sizes of the LFR benchmark datasets. For each dataset, 5 different epsilon values were tested (0.2, 0.4, 0.6, 0.8, 1.0). The epsilon value that resulted in the highest modularity was then selected to evaluate clustering accuracy. To evaluate the accuracy of predicted clusters, we used the Adjust Rand Index (ARI) and the Normalized Mutual Information (NMI). After running PSCAN on all datasets and epsilon values, we get the modularity scores from the resulting clusters in the Table III. Using epsilon = 0.6 resulted in the highest modularity across for datasets except LFR_160000 which had the highest modularity for epsilon = 0.4. It isn't clear whether the authors used the same epsilon for all datasets, but we will use epsilon = 0.6 for all datasets when computing the ARI and NMI for consistency. The authors also did not say which epsilon value they chose so these results are not guaranteed to be one-to-one. When using epsilon = 0.6 to evaluate ARI and NMI we receive the results in the Table IV.

2) *Runtime Performance*: In order to evaluate runtime performance, we ran PSCAN on BA graphs, keeping the epsilon value as a constant and increasing the number of machines to

TABLE IV

COMPARISON OF PAPER AND REPRODUCED ARI/NMI RESULTS AT $\epsilon = 0.6$.

Dataset	Paper ARI	Our ARI	Paper NMI	Our NMI
lfr_5000	0.997	0.988	0.999	0.997
lfr_10000	0.998	0.990	0.999	0.998
lfr_20000	0.969	0.977	0.977	0.994
lfr_40000	0.981	0.977	0.988	0.995
lfr_80000	0.971	0.976	0.987	0.995
lfr_160000	0.999	0.995	0.999	0.998

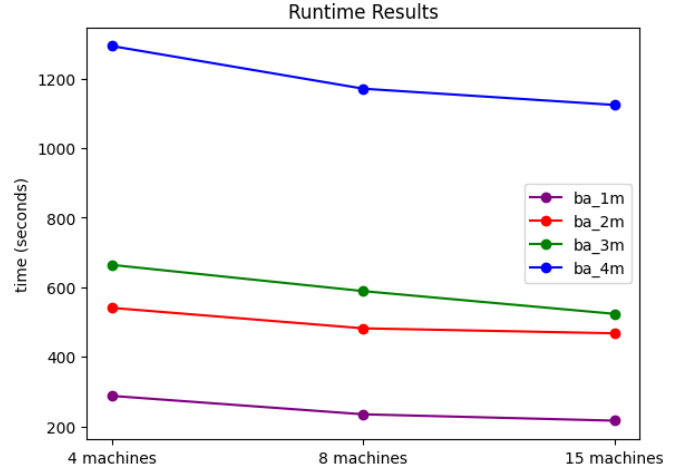


Fig. 1. Runtime Analysis across different numbers of CPU cores.

measure speedup. We evaluated the runtime of the algorithm using 4, 8 and 15 machines using the following equation.

$$\text{Relative speedup}(m) = \frac{\text{running time on 4 computers}}{\text{running time on } m \text{ computers}}$$

In the Figure 1, you can see the absolute time in seconds it took to run PSCAN on all BA graphs using 4, 8, and 15 machines. We used more powerful machines than those of the author, and as a result, our time in absolute seconds is significantly lower than that of the original paper. By converting these results to speedup values as shown in the Figure 2, we can compare the effect of increasing the number of CPU cores relative to the minimum number of CPU cores. We also observe improvement in the runtime with the increase in the number of CPU cores.

3) *Orkut Network Clustering Evaluation*: For the Orkut dataset, we evaluated PSCAN cluster quality using modularity on the original graph. Among the tested thresholds, $\epsilon = 0.4$ achieved the highest modularity score of 0.1091, while larger thresholds led to a substantial drop in modularity. This suggests that moderate pruning preserves useful structural communities, whereas overly strict similarity thresholds fragment or distort the clustering structure. The modularity results on the original Orkut network for different epsilon values are given in the Table V.

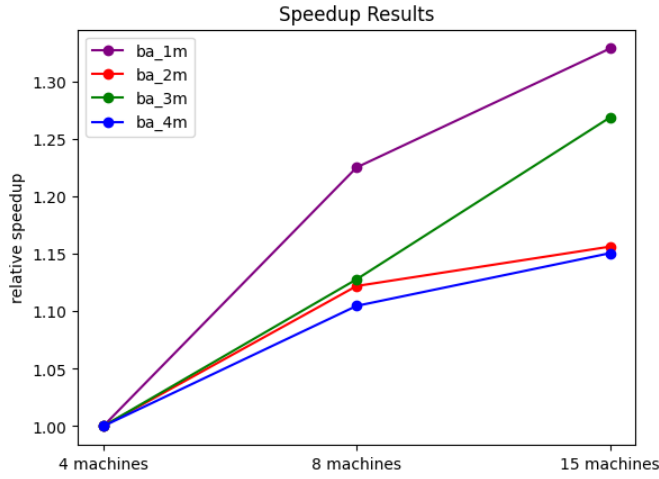


Fig. 2. Speedup Analysis across different numbers of CPU cores.

TABLE V
ORKUT MODULARITY RESULTS UNDER DIFFERENT ϵ VALUES FOR THE ORIGINAL ORKUT NETWORK.

Epsilon	Number of Clusters	Modularity
0.2	54,459	0.089952
0.4	141,166	0.109144
0.6	58,236	0.007115
0.8	6,848	0.000133
1.0	771	0.000008

V. CONCLUSION

This project satisfies the three main objectives of the course project brief. First, it provides a clear explanation of the PSCAN paper and its main algorithmic ideas, including structural similarity computation, edge pruning, hub/outlier detection, and label-propagation-based connected-component clustering. Second, it implements the major PSCAN components in a modular Python pipeline, together with reproducible dataset-preparation utilities for synthetic and real graphs. Third, it evaluates the resulting implementation experimentally using benchmark and real-world datasets, thereby addressing both clustering quality and runtime behavior.

From the clustering-accuracy perspective, the reproduced results on LFR graphs are strong and broadly consistent with the original paper. Across the six benchmark datasets, our ARI values range from 0.977 to 0.995 and our NMI values range from 0.994 to 0.998, which are close to the paper’s reported values and confirm that the implementation is capturing the intended clustering behavior of PSCAN. The modularity results are also consistently high across the tested ϵ values, indicating that PSCAN identifies structurally coherent clusters on the synthetic benchmarks. These results support the claim that PSCAN is effective for clustering large graphs with planted community structure.

From the scalability perspective, our runtime experiments on Barabási graphs show that the implementation benefits from increased compute resources. The experiments demonstrate the expected trend that larger graph-processing workloads can benefit from additional parallel resources.

In summary, the project delivers a meaningful reproduction of PSCAN that combines theoretical understanding, practical implementation, and empirical evaluation. The results show that PSCAN remains a compelling approach for structural clustering of large networks, especially when clustering quality is the primary objective. Future work could focus on a closer distributed reimplement of the original Hadoop pipeline, stronger evaluation on overlapping real-world community data, and further optimization of the communication and I/O bottlenecks in our experiments.

REFERENCES

- [1] Weizhong Zhao, Venkataswamy Martha, and Xiaowei Xu. Pscan: A parallel structural clustering algorithm for big networks in mapreduce. In *2013 IEEE 27th International Conference on Advanced Information Networking and Applications (AINA)*, pages 862–869, 2013.
- [2] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. In Gaël Varoquaux, Travis Vaught, and Jarrod Millman, editors, *Proceedings of the 7th Python in Science Conference*, pages 11 – 15, Pasadena, CA USA, 2008.
- [3] Jaewon Yang and Jure Leskovec. Defining and evaluating network communities based on ground-truth, 2012.